

Lab 03 — Hands-on lab: life, signals and death of a container

Goal: trigger every behaviour from the theory yourself — the immediate exit, the 10 seconds of agony, code 137, automatic restart — and see who watches over your containers when there is no daemon.

Prerequisites — Labs 01 and 02 completed. Images `alpine` and `nginx:alpine` present.

Files provided — `files/demarrage-casse.sh` (broken start-up script) and `files/demarrage-correct.sh` (correct start-up script), used in step 3.

Step 1 — The states, one by one

```
podman create --name state alpine sleep 120
podman ps -a --filter name=state --format 'table {{.Names}}\t{{.Status}}'
```

Observe the status `Created`: the container exists, but no process is running yet.

```
podman start state
podman ps --filter name=state --format '{{.Status}}'
podman pause state && podman ps -a --filter name=state --format '{{.Status}}'
podman unpause state && podman ps --filter name=state --format '{{.Status}}'
podman stop -t 2 state && podman ps -a --filter name=state --format '{{.Status}}'
podman start state && podman ps --filter name=state --format '{{.Status}}'
podman rm -f -t 0 state
```

Observe the sequence: `Up 1 second`, `Paused`, `Up 5 seconds`, a warning `StopSignal SIGTERM failed to stop container state in 2 seconds, resorting to SIGKILL`, `Exited (137)`, then `Up` again.

Explanation. An `Exited` container can be started again: it kept its configuration and its writable layer. Also note that `podman ps` without `-a` **does not show** a paused container — it is not "running".

Step 2 — Why a container stops on its own

```
podman run --name try1 alpine
podman ps -a --filter name=try1 --format '{{.Status}}'
```

Observe `Exited (0)`: the default command for `alpine` is `/bin/sh`, and with no input it exits immediately.

```
podman run -d --name try2 nginx:alpine
podman ps --filter name=try2 --format '{{.Status}}'
```

Observe `Up`: nginx runs in the foreground.

```
podman run --rm alpine sh -c 'sleep 60 & echo "started in the background"'
```

Observe that the command returns **immediately**, even though a `sleep 60` really was started.

Explanation. The `&` detached `sleep`; the shell ran `echo` and exited. With PID 1 gone, the container was destroyed and took `sleep` with it. This is the single most common start-up-script mistake.

Look at who is watching `try2` while it runs:

```
podman inspect --format '{{.State.CommonPid}}' try2
ps -o pid,ppid,user,comm -p $(podman inspect --format '{{.State.CommonPid}}' try2)
```

Observe a `common` process, running under **your** user: this is the supervisor Podman leaves behind each container — the only "daemon" you have left, and it uses just a few hundred KB.

```
podman rm try1 ; podman rm -f -t 0 try2
```

Step 3 — The broken start-up script, and its fix

Copy the two scripts provided:

```
mkdir -p ~/labo-docker/03 && cd ~/labo-docker/03
cp <lab-path>/files/*.sh . && chmod +x *.sh
cat demarrage-casse.sh demarrage-correct.sh
```

Run the first one **inside** a container, mounting the current folder:

```
podman run --rm -v "$PWD":/scripts alpine /scripts/demarrage-casse.sh
```

Observe: the message appears and your prompt comes straight back — the container is already dead and removed.

```
podman run -d --name correct -v "$PWD":/scripts alpine /scripts/demarrage-correct.sh
podman ps --filter name=correct --format '{{.Status}}'
podman top correct
```

Observe that the container stays `Up`, and that `podman top` shows `sleep 300` as PID 1 — with **no** parent `sh` process.

Explanation. The `exec` in the second script **replaced** the shell with the final command, which inherits PID 1. This is exactly what the `exec` form of an `ENTRYPOINT` does; lab 04 covers it.

→ LINUX

`exec` is a shell built-in that invokes the system call of the same name: the current process drops its program (the shell) and loads the requested program **in its place**, keeping its PID. Without `exec`, the shell creates a child (`fork`) and waits. With `exec`, no shell remains at all.

```
podman rm -f -t 0 correct
```

Step 4 — The 10 seconds of agony

Time a stop on a process that ignores `SIGTERM`:

```
podman run -d --name idle alpine sleep 300
time podman stop idle
podman inspect --format 'code={{.State.ExitCode}} oom={{.State.OOMKilled}}' idle
podman rm idle
```

Observe the warning `StopSignal SIGTERM failed to stop container idle in 10 seconds, resorting to SIGKILL`, `real 0m10.1s`, and `code=137 oom=false`.

Try again with a mini-init:

```
podman run -d --init --name idle alpine sleep 300
podman exec idle ps -o pid,comm
time podman stop idle
podman inspect --format 'code={{.State.ExitCode}}' idle
podman rm idle
```

Observe `1 podman-init` with `sleep` below it as PID 2, a stop in `0m0.1s`, and `code=143`.

And with an application that handles its signals properly:

```
podman run -d --name web nginx:alpine
time podman stop web
podman inspect --format 'code={{.State.ExitCode}}' web
podman rm web
```

Observe an instant stop and `code=0`.

Explanation. Three behaviours, three causes. `sleep` as PID 1 **ignores** `SIGTERM` (kernel protection): the engine waits, then kills → `137`. With `--init`, `sleep` is no longer PID 1, so the default action applies → `143`. `nginx` installs a signal handler and shuts down cleanly (with code `0`, because `nginx` chooses to exit normally). Multiply those ten seconds by the number of containers you run, and you know where the mystery delay in your redeployments comes from.

You can shorten the grace period — though it does not fix the cause:

```
podman run -d --name idle alpine sleep 300
time podman stop -t 2 idle
podman rm idle
```

Observe `real 0m2.1s`, still with code `137`.

Step 5 — Read exit codes

```
podman run --rm alpine sh -c 'exit 0' ; echo "code=$?"
podman run --rm alpine sh -c 'exit 3' ; echo "code=$?"
podman run --rm alpine missing-command ; echo "code=$?"
```

Observe `0`, `3`, then `127` together with `Error: crun: executable file missing-command not found in $PATH`.

```
podman run -d --name killed alpine sleep 300
podman kill killed
podman inspect --format 'code={{.State.ExitCode}}' killed
podman rm killed
```

Observe `137`, with no waiting this time: `kill` has no grace period.

Now trigger a genuine out-of-memory kill:

```
podman run --name oom --memory=32m --memory-swap=32m alpine sh -c 'head -c 100m /dev/zero |
tail'
echo "code=$?"
podman inspect --format 'code={{.State.ExitCode}} oom={{.State.OOMKilled}}' oom
podman rm oom
```

Observe `code=137`, but this time `oom=true`: same code, different cause — and only `inspect` can tell them apart.

Explanation. Above 128, the code means death by signal: `code - 128` gives the signal number. `127`, by contrast, is a launch error: the application never started at all.

Step 6 — `exec` versus `attach`

```
podman run -d --name web nginx:alpine
podman exec web nginx -v
podman exec -it web sh
```

In the shell you get, type:

```
ps -o pid,comm
exit
```

Observe that `nginx` is PID 1, followed by its *workers*, and that your `sh` has a different PID. After you leave the shell, the container is **still** `Up`.

```
podman ps --filter name=web --format '{{.Status}}'
```

Explanation. `exec` created a **new** process in the container's namespaces; leaving it does not affect PID 1. `attach`, by contrast, would hook you onto `nginx` itself — one `Ctrl+C` and it stops. To read logs, always use:

```
podman logs --tail 5 web
podman logs -f --since 1m web      # Ctrl+C here only stops the display
```

Step 7 — Logs only come from `stdout`

```
podman run --rm --name logs-demo alpine sh -c \  
  'echo "I go to stdout"; echo "I go to a file" > /tmp/app.log; sleep 1'
```

Observe that only the first line appears.

```
podman run -d --name logs-demo alpine sh -c \  
  'echo "visible"; echo "invisible" > /tmp/app.log; sleep 120'  
podman logs logs-demo  
podman exec logs-demo cat /tmp/app.log  
podman rm -f -t 0 logs-demo
```

Observe that `podman logs` prints only `visible`; you can reach the file's content only by going inside the container.

Explanation. `common` captures only `stdout` and `stderr` of PID 1. That is why a containerised application must log to the console — and why you must not configure `logging.file.name` in a containerised Spring Boot.

Step 8 — Automatic restart

```
podman run -d --restart=on-failure:3 --name unstable alpine \  
  sh -c 'echo "start $(date +%T)"; sleep 3; exit 1'  
sleep 20  
podman ps -a --filter name=unstable --format '{{.Names}} {{.Status}}'  
podman inspect --format 'restarts={{.RestartCount}} code={{.State.ExitCode}}' unstable  
podman logs unstable
```

Observe a `RestartCount` of `3`, a status of `Exited (1)`, and **four** "start" lines in the logs: the initial attempt plus three retries.

Explanation. Logs from successive runs accumulate on the same container: the first line holds the root cause. `.State`, on the other hand, describes only the **last** run.

```
podman rm unstable  
podman events --since 2m --until 1s | grep unstable | awk '{print $5, $6}' | uniq -c
```

Observe the event journal: `container start`, `container died`, `container restart`... This is the only place where you see a container's *history*, not just its current state.

Finally, confirm the conflict the theory announced:

```
podman run --rm -d --restart=always nginx:alpine
```

Observe `Error: the --rm option conflicts with --restart, when the restartPolicy is not "" and "no"`.

PODMAN

What about a reboot? Test it yourself: run `podman run -d --restart=always --name survivor nginx:alpine`, close **all** your Ubuntu windows, and from PowerShell run `wsl --shutdown`. Reopen Ubuntu: `podman ps` is empty. Nobody re-read the policy — there is no daemon. On a server, `systemd` takes over that role through a Quadlet file (lab 10). On your workstation this is fine: development containers do not need to survive a reboot. Clean up afterwards with `podman rm -f -t 0 survivor`.

Step 9 — Extract evidence from a dead container

```
podman run --name autopsy alpine sh -c 'echo "important trace" > /report.txt; exit 2'
podman ps -a --filter name=autopsy --format '{{.Status}}'
podman cp autopsy:/report.txt ./report.txt
cat report.txt
```

Observe that you can recover the file even though the container shows `Exited (2)`.

```
podman rm autopsy
podman cp autopsy:/report.txt ./other.txt
```

Observe `Error: container "autopsy" does not exist`: once the container is removed, everything is gone.

Explanation. The operations rule: **inspect before you remove**. `cp`, `logs`, and `inspect` work on a stopped container — never on a removed one.

Clean-up

```
podman ps -a --format 'table {{.Names}}\t{{.Status}}'
podman rm -f -t 0 idle web correct unstable autopsy try1 try2 killed state oom survivor
2>/dev/null
rm -f ~/labo-docker/03/report.txt
podman ps -a --format '{{.Names}}'
```

Observe that no container from this lab remains. The images `alpine` and `nginx:alpine` are kept.

What you must be able to state now

- A container dies with its PID 1 — you triggered all three cases yourself.
- `sleep` as PID 1 ignores `SIGTERM`; `--init` fixes the symptom, `exec` fixes the cause.
- `137` = killed (by `stop`, `kill`, or the OOM killer — `inspect` settles it), `143` = stopped cleanly, `127` = command not found.
- `exec` creates a process, `attach` hooks onto PID 1.
- `podman logs` shows only `stdout` / `stderr`, as captured by `conmon`.
- `podman rm` destroys logs and evidence: inspect first.
- Without a daemon, a *restart policy* does not survive a `wsl --shutdown`.